TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM

KHOA KỸ THUẬT MÁY TÍNH

IT012 – TỔ CHỨC VÀ CẤU TRÚC MÁY TÍNH II

LAB 04

MẢNG, TRUY XUẤT BỘ NHỚ VÀ LỆNH ĐIỀU KHIỂN

Họ và tên	Nguyễn Công Danh	MSSV	25520249
Lớp	IT012.R11.2	Ngày thực hành	09/11/2026
Tự chấm	11/10	Link git	Link git

1. Tổng quan buổi thực hành

Mục đích. Buổi thực hành này tiếp nối LAB 03. Ở LAB 03, dữ liệu nằm trong thanh ghi và chương trình chạy một mạch từ lệnh đầu đến lệnh cuối. Chương trình thực tế phải xử lý nhiều dữ liệu hơn số thanh ghi hiện có nên dữ liệu được đặt trong bộ nhớ, và chương trình phải biết rẽ nhánh theo điều kiện và lặp lại một đoạn lệnh. Sinh viên sẽ lưu một dãy số trong mảng, đọc và ghi từng phần tử bằng `lw` và `sw`, dùng lệnh rẽ nhánh để ra quyết định và dùng vòng lặp để duyệt toàn bộ mảng. Mỗi khái niệm mới đều được giải thích trước rồi mới thực hành.

> **Tình huống: bộ nhớ của công tơ điện thông minh**
>
> Công tơ điện thông minh không chỉ lưu một chỉ số mà lưu chỉ số cuối mỗi tháng trong bộ nhớ: 13 chỉ số liên tiếp (`reading[0]` đến `reading[12]`) cho 12 tháng. Điện năng tiêu thụ của một tháng là hiệu của hai chỉ số liên tiếp. Phần mềm nhúng của công tơ phải lấy từng chỉ số từ bộ nhớ, xác định mức tiêu thụ thuộc bậc giá nào, rồi duyệt cả năm để tính tổng điện năng và tìm tháng tiêu thụ nhiều nhất. Mọi chỉ số và bảng bậc giá trong bài thực hành này đều là số liệu giả định.

Sau buổi thực hành, sinh viên có thể:

- Khai báo mảng trong phần `.data` và xác định địa chỉ của một phần tử bằng địa chỉ nền cộng độ dời (bội của 4);
- Dùng `lw` và `sw` để đọc, ghi phần tử của mảng, và giải thích vì sao việc tính toán phải đi qua thanh ghi;
- Chuyển cấu trúc if-else sang các lệnh `beq`, `bne`, `slt` và `j`, và nhận biết cách assembler mở rộng lệnh giả rẽ nhánh;
- Đọc mã máy của lệnh rẽ nhánh và kiểm tra địa chỉ đích theo công thức trong bài giảng;
- Viết vòng lặp duyệt mảng, tính địa chỉ phần tử bằng `sll` và `add`, và gỡ lỗi bằng breakpoint, Step và Backstep;
- Dự đoán, quan sát và giải thích hành vi của một chương trình hợp ngữ có rẽ nhánh và vòng lặp.

Phần	Nội dung	Điểm
Phần 0	Chuẩn bị môi trường	–
TN 1	Thí nghiệm 1 – Bộ nhớ và mảng	2
TN 2	Thí nghiệm 2 – Lệnh điều khiển	2,5
TN 3	Thí nghiệm 3 – Duyệt mảng bằng vòng lặp	4
MR	Mở rộng – Đếm số tháng theo bậc giá	+1
Kết	Tổng kết và nộp bài	1,5
Tổng		11

Điểm phần mở rộng được cộng vào tổng điểm của buổi thực hành; tổng điểm tối đa là 11.

Quy ước làm bài

• Ô nền vàng là chỗ để ghi câu trả lời hoặc giá trị; chữ gõ vào ô nền vàng hiển thị màu đỏ. Khung nét đứt là chỗ để dán ảnh chụp màn hình.

• Mã nguồn được đặt trong khung màu xám, font đơn cách. Trong mã nguồn, mọi nội dung sau dấu # là chú thích. Dòng bắt đầu bằng # >>> đánh dấu chỗ sinh viên tự viết mã.

• Mỗi câu hỏi được gắn nhãn theo mức nhận thức mà câu đó yêu cầu: *Hiểu*, *Vận dụng*, *Phân tích* hoặc *Đánh giá*.

• Nhiều hoạt động theo chu trình **DỰ ĐOÁN → QUAN SÁT → GIẢI THÍCH**. Trước hết ghi lại điều mình mong đợi (DỰ ĐOÁN), sau đó chạy chương trình và ghi lại điều thực sự xảy ra (QUAN SÁT), cuối cùng giải thích sự khác biệt (GIẢI THÍCH). Dự đoán sai không bị trừ điểm; điểm nằm ở phần giải thích.

• Đánh dấu vào ô ☐ trong danh sách kiểm tra khi đã hoàn thành mục tương ứng.

PHẦN 0 — Chuẩn bị môi trường

Các thao tác cơ bản trên MARS đã được làm quen ở LAB 03. Bảng 2 liệt kê các thao tác mới cần dùng trong bài này; đánh dấu vào ô ☐ khi đã thực hiện được.

Bảng 2. Danh sách kỹ năng

☑	Kỹ năng	Gợi ý
☑	Kiểm tra các thiết lập dùng trong bài này	Trong menu Settings: bật *Permit extended (pseudo) instructions and formats*; bật *Initialize Program Counter to global 'main' if defined*; tắt *Delayed branching*; tắt *Popup dialog for input syscalls*. Giữ nguyên cấu hình bộ nhớ mặc định.
☑	Xem Data Segment ở dạng hexa hoặc thập phân	Dưới khung Data Segment, đánh dấu hoặc bỏ đánh dấu ô *Hexadecimal Values*. Mỗi hàng hiển thị 8 từ nhớ liên tiếp; cột *Address* là địa chỉ của từ đầu hàng.
☑	Xem địa chỉ của các nhãn	Settings → Show Labels Window (symbol table).
☑	Đặt breakpoint	Trong tab Execute, đánh dấu ô *Bkpt* cạnh một lệnh trong Text Segment. Run → Go dừng trước lệnh đó; chạy tiếp bằng Run → Go hoặc Run → Step.

Bước 0 – Kiểm tra nhanh

Gõ chương trình ở Mã nguồn 0 vào một tệp mới (gõ đúng nguyên văn), lưu lại, assemble và chạy. Chương trình đọc phần tử thứ ba của một mảng nhỏ rồi in ra; ý nghĩa từng dòng được giải thích trong Thí nghiệm 1.

Mã nguồn 0

```
# Ma nguon 0 - kiem tra nhanh
.data
arr: .word 7, 5, 12
msg: .asciiz "Phan tu arr[2] = "

.text
.globl main
main:
  la  $s0, arr
  lw  $t0, 8($s0)
  li  $v0, 4
  la  $a0, msg
  syscall
  li  $v0, 1
  add $a0, $t0, $zero
  syscall
  li  $v0, 10
  syscall
```

Văn bản xuất hiện trong khung Run I/O:

```
Phan tu arr[2] = 12
```

> **Quy tắc đặt tên tệp**
> Mỗi chương trình nộp bài được lưu với tên `IT012_Lab04_<MSSV>_<Phần>.asm`, ví dụ `IT012_Lab04_22520001_TN2.asm`. Dùng TN1, TN2, TN3 hoặc MR cho phần tương ứng. Đầu mỗi tệp phải có một khối chú thích gồm họ tên, MSSV và mục đích của chương trình trong một dòng.

THÍ NGHIỆM 1. BỘ NHỚ VÀ MẢNG (2 điểm)

Tình huống

Công tơ phải nhớ 13 chỉ số cuối tháng. Số lượng này vượt xa số thanh ghi nên các chỉ số nằm trong bộ nhớ. Để tính toán, bộ xử lý nạp từng số vào thanh ghi, tính, rồi ghi kết quả trở lại bộ nhớ.

Kiến thức nền: bộ nhớ, mảng và các lệnh lw, sw

Bộ nhớ được đánh địa chỉ theo byte. Một từ (word) gồm 4 byte liên tiếp nên địa chỉ của một từ phải là bội của 4. Trong MARS, phần `.data` bắt đầu tại địa chỉ `0x10010000`.

Chỉ thị `.word a, b, c` đặt các số 32 bit liên tiếp trong bộ nhớ; `.space n` chừa `n` byte trống. Nhãn là địa chỉ của phần tử đầu tiên, gọi là *địa chỉ nền* của mảng. Phần tử thứ k của một mảng các từ nằm tại địa chỉ nền + 4k.

`lw rt, độ_dời(rs)` nạp từ nhớ tại địa chỉ `rs + độ_dời` vào `rt`; `sw rt, độ_dời(rs)` ghi `rt` vào từ nhớ đó. Độ dời (offset) là một hằng số. Ví dụ, nếu `$s0` chứa địa chỉ nền của mảng A thì `lw $t0, 20($s0)` nạp A[5] vào `$t0`. Các lệnh như `add` chỉ làm việc với thanh ghi, nên phải nạp dữ liệu trước rồi mới tính.

Bước 1.1 – Mảng trong bộ nhớ

Gõ Mã nguồn 1, lưu và assemble nhưng chưa chạy; bật hiển thị hexa cho Data Segment và mở Labels window. Điền cột DỰ ĐOÁN của Bảng 3 (địa chỉ nền là `0x10010000`; phần tử thứ k ở nền + 4k), rồi điền cột QUAN SÁT từ cột *Address* của Data Segment (nhãn thì xem Labels window) và giá trị thập phân của phần tử.

Mã nguồn 1

```
# Ma nguon 1 - mang trong bo nho
.data
reading: .word 1000, 1095, 1205, 1345, 1510, 1715, 1955
         .word 2210, 2440, 2625, 2775, 2895, 2995
usage:   .space 48

.text
.globl main
main:
  li  $v0, 10
  syscall
```

Bảng 3. Địa chỉ và giá trị của các phần tử

Phần tử	Địa chỉ DỰ ĐOÁN (hexa)	Địa chỉ QUAN SÁT (hexa)	Giá trị (thập phân)
`reading[0]`	0x10010000	0x10010000	1000
`reading[1]`	0x10010004	0x10010004	1095
`reading[3]`	0x1001000c	0x1001000c	1345
`reading[12]`	0x10010030	0x10010030	2995
nhãn `usage`	0x10010034	0x10010034	–

Câu 1.1 (Hiểu) Giải thích vì sao địa chỉ của `reading[3]` cách địa chỉ của `reading[0]` đúng 12 đơn vị chứ không phải 3.

Bộ nhớ kiến trúc MIPS được đánh địa chỉ theo từng byte. Mỗi phần tử kiểu word (32-bit) chiếm 4 byte liên tiếp trong bộ nhớ. Phần tử reading[3] là phần tử thứ 3 tính từ phần tử đầu tiên reading[0], do đó độ lệch địa chỉ bằng 3 x 4 = 12 byte chứ không phải 3.

Bước 1.2 – Đọc và ghi bộ nhớ bằng lw, sw

Giữ phần `.data` của Mã nguồn 1 và thay phần `.text` bằng Mã nguồn 2. Chương trình đã tính điện năng tháng 1 và ghi vào `usage[0]`. Hoàn thành chương trình cho tháng 2 và tháng 3 (ghi vào `usage[1]` và `usage[2]`). Điền cột DỰ ĐOÁN của Bảng 4 trước khi chạy; sau đó chạy chương trình và đọc giá trị trong Data Segment.

Mã nguồn 2

```
# Ma nguon 2 - doc reading, ghi usage
.text
.globl main
main:
  la  $s0, reading      # $s0 = dia chi nen cua reading
  la  $s1, usage        # $s1 = dia chi nen cua usage
  lw  $t0, 0($s0)       # $t0 = reading[0]
  lw  $t1, 4($s0)       # $t1 = reading[1]
  sub $t2, $t1, $t0     # $t2 = dien nang thang 1
  sw  $t2, 0($s1)       # usage[0] = $t2
  # >>> dien nang thang 2 -> usage[1]
  # >>> dien nang thang 3 -> usage[2]
  li  $v0, 10
  syscall
```

Bảng 4. Điện năng các tháng ghi vào mảng usage

Phần tử	Địa chỉ (hexa)	DỰ ĐOÁN (thập phân)	QUAN SÁT (thập phân)
`usage[0]`	0x10010034	95	95
`usage[1]`	0x10010038	110	110
`usage[2]`	0x1001003c	140	140

Câu 1.2 (Hiểu) Với mỗi tháng, chương trình dùng bao nhiêu lệnh truy cập bộ nhớ và bao nhiêu lệnh tính toán? Từ đó giải thích vì sao MIPS được gọi là kiến trúc nạp–lưu (load–store).

Với mỗi tháng, chương trình sử dụng 3 lệnh truy cập bộ nhớ cùng 1 lệnh tính toán.
MIPS được gọi là kiến trúc nạp - lưu vì các phép tính toán chỉ có thể thực hiện trực tiếp trên các thanh ghi bên trong CPU; kiến trúc không cho phép tính toán trực tiếp trên các ô nhớ mà bắt buộc phải nạp dữ liệu từ bộ nhớ vào thanh ghi trước, thực hiện tính toán, rồi lưu kết quả từ thanh ghi ngược lại bộ nhớ.

Bước 1.3 – Cố ý làm sai

Với mỗi thí nghiệm trong Bảng 5, DỰ ĐOÁN điều sẽ xảy ra, thực hiện thay đổi, chạy chương trình và QUAN SÁT kết quả. Khôi phục chương trình sau mỗi thí nghiệm.

Bảng 5. Các thí nghiệm có kiểm soát

Thí nghiệm	DỰ ĐOÁN	QUAN SÁT
(a) Trong Mã nguồn 2, đổi `lw $t1, 4($s0)` thành `lw $t1, 2($s0)`.	Chương trình xảy ra lỗi ngoại lệ do địa chỉ không căn chỉnh đúng biên của một từ	fetch address not aligned on word boundary 0x10010002
(b) Trong Mã nguồn 2, đổi `sw $t2, 0($s1)` thành `sw $t2, 0($s0)`, rồi xem `reading[0]` trong Data Segment.	Ô nhớ reading[0] bị ghi đè giá trị mới bằng 95 thay vì 1000	Giá trị tại địa chỉ 0x10010000 (reading[0]) bị đổi từ 1000 (0x000003e8) thành 95 (0x0000005f)

Câu 1.3 (Phân tích) Giải thích quan sát ở (a) và (b). Khi ghi vào bộ nhớ, bộ xử lý có kiểm tra ô nhớ đó có phải là chỗ dành cho kết quả hay không?

Ở thí nghiệm (a): Lệnh lw trong kiến trúc MIPS yêu cầu địa chỉ truy xuất phải chia hết cho 4. Địa chỉ nền 0x10010000 + 2 = 0x10010002 không chia hết cho 4 nên phần cứng bẫy lỗi ngoại lệ căn chỉnh địa chỉ.
Ở thí nghiệm (b): Địa chỉ 0x10010000 hoàn toàn hợp lệ trong phân vùng dữ liệu và được căn chỉnh đúng từ, nên CPU thực hiện ghi dữ liệu bình thường.
Kết luận: Khi ghi vào bộ nhớ, bộ xử lý không kiểm tra ô nhớ đó có phải là nơi dành cho kết quả hay không; CPU chỉ kiểm tra tính hợp lệ về mặt địa chỉ, chứ không phân biệt được ý nghĩa logic của dữ liệu.

THÍ NGHIỆM 2. LỆNH ĐIỀU KHIỂN (2,5 điểm)

Tình huống

Công ty điện lực tính giá điện theo bậc thang: tiêu thụ càng nhiều thì thuộc bậc càng cao (Bảng 6). Để xác định bậc, công tơ phải so sánh điện năng với các ngưỡng và chọn một trong nhiều đường thực thi, tức là phải rẽ nhánh.

Bảng 6. Bậc giá điện (số liệu giả định)

Bậc	Điện năng tiêu thụ trong tháng (kWh)
1	từ 0 đến 100
2	từ 101 đến 150
3	từ 151 đến 200
4	từ 201 trở lên

Kiến thức nền: rẽ nhánh và nhảy

Mặc định bộ xử lý thực hiện lệnh kế tiếp. Lệnh rẽ nhánh có điều kiện `beq rs, rt, nhãn` (branch if equal) nhảy đến nhãn nếu `rs` bằng `rt`; `bne` (branch if not equal) nhảy nếu `rs` khác `rt`; nếu điều kiện sai thì thực hiện lệnh kế tiếp. Lệnh `j nhãn` nhảy không điều kiện.

MIPS không có lệnh "nhảy nếu nhỏ hơn". Để so sánh nhỏ hơn, dùng `slt rd, rs, rt` (`rd` = 1 nếu `rs < rt`, ngược lại `rd` = 0) hoặc `slti rd, rs, hằng_số`, rồi dùng `bne` hoặc `beq` với `$zero`. Ví dụ, nhảy đến nhãn `L` nếu `$s0 < $s1`: `slt $t0, $s0, $s1` rồi `bne $t0, $zero, L`. Các lệnh giả `blt`, `bgt`, `ble`, `bge` cho phép viết gọn hơn; assembler tự mở rộng chúng thành lệnh thật.

Bước 2.1 – Đi theo đường rẽ nhánh (DỰ ĐOÁN, QUAN SÁT)

Mã nguồn 3 là bản dịch của câu lệnh `if (i == j) f = g + h; else f = g - h;` với f, g, h, i, j lần lượt là `$s4`, `$s2`, `$s3`, `$s0`, `$s1`. Gõ chương trình, assemble, rồi với mỗi trường hợp trong Bảng 7: DỰ ĐOÁN giá trị cuối cùng của f và lệnh được thực thi ngay sau `bne`, sau đó chạy bằng Step để QUAN SÁT. Ở trường hợp thứ hai, đổi lệnh `addi $s1, $zero, 4` thành `addi $s1, $zero, 5`.

Mã nguồn 3

```
# Ma nguon 3 - if / else
.text
.globl main
main:
  addi $s0, $zero, 4     # i = 4
  addi $s1, $zero, 4     # j = 4
  addi $s2, $zero, 10    # g = 10
  addi $s3, $zero, 3     # h = 3
  bne  $s0, $s1, Else    # neu i != j thi nhay den Else
  add  $s4, $s2, $s3     # f = g + h
  j    Exit
Else:
  sub  $s4, $s2, $s3     # f = g - h
Exit:
  li   $v0, 10
  syscall
```

Bảng 7. Hai đường thực thi của if-else

Trường hợp	f DỰ ĐOÁN	f QUAN SÁT	Lệnh sau bne DỰ ĐOÁN	Lệnh sau bne QUAN SÁT
i = 4, j = 4	13	13	add $s4, $s2, $s3	add $s4, $s2, $s3
i = 4, j = 5	7	7	sub $s4, $s2, $s3	sub $s4, $s2, $s3

Câu 2.1 (Hiểu) Với i = j, điều gì xảy ra nếu xóa lệnh `j Exit`? Giải thích bằng giá trị của f.

Nếu xóa lệnh j Exit, khi i = j (nhánh If đúng), CPU thực hiện lệnh add $s4, $s2, $s3 khiến f = 10 + 3 = 13. Sau đó, do không có lệnh nhảy qua khối Else, CPU sẽ tiếp tục chạy tuần tự xuống dòng kế tiếp là sub $s4, $s2, $s3 tại nhãn Else:, dẫn đến f bị ghi đè thành 10 - 3 = 7. Kết quả cuối cùng f sẽ bằng 7 thay vì 13.

Bước 2.2 – Lệnh giả rẽ nhánh

Gõ Mã nguồn 4, assemble (chưa chạy) và quan sát Text Segment. Cột *Source* là dòng đã gõ, cột *Basic* là các lệnh thật mà bộ xử lý thực thi. Ghi lại các dòng *Basic* ứng với `blt` vào Bảng 8. Sau đó đổi `blt` thành `bge`, assemble lại và ghi các dòng *Basic* mới.

Mã nguồn 4

```
# Ma nguon 4 - gia lenh nhanh
.text
.globl main
main:
  addi $t0, $zero, 90
  addi $t1, $zero, 100
  blt  $t0, $t1, Nho
  addi $t2, $zero, 1
Nho:
  li   $v0, 10
  syscall
```

Bảng 8. Lệnh giả được mở rộng

Dòng *Source*	Các lệnh thật (cột *Basic*)	Số lệnh thật
`blt $t0, $t1, Nho`	slt $at, $t0, $t1 bne $at, $zero, Nho	2
`bge $t0, $t1, Nho`	slt $at, $t0, $t1 beq $at, $zero, Nho	2

Câu 2.2 (Phân tích) Assembler dùng thanh ghi nào làm trung gian khi mở rộng `blt`, và vì sao cần thanh ghi đó? Hai lệnh thật của `blt` và `bge` khác nhau ở điểm nào?

Assembler sử dụng thanh ghi $at làm trung gian. Cần thanh ghi này vì MIPS theo triết lý phần cứng đơn giản, không có lệnh phần cứng so sánh và rẽ nhánh trực tiếp cho phép toán nhỏ hơn/lớn hơn trong một chu kỳ; assembler phải tách thành một lệnh so sánh lưu cờ trạng thái tạm thời vào $at, sau đó dùng lệnh rẽ nhánh kiểm tra giá trị của $at.

Điểm khác nhau: Cả hai đều dùng slt $at, $t0, $t1 (gán $at = 1 nếu $t0 < $t1, ngược lại $at = 0), nhưng blt dùng lệnh bne $at, $zero, Nho, còn bge dùng lệnh beq $at, $zero, Nho.

Bước 2.3 – Mã máy của lệnh rẽ nhánh

Lệnh rẽ nhánh thuộc định dạng I (`op | rs | rt | immediate`, 16 bit cuối là *immediate*). Theo bài giảng, địa chỉ đích của lệnh rẽ nhánh được tính bằng `PC + 4 + (immediate × 4)`. Quay lại Mã nguồn 3 (đã assemble) và hoàn thành Bảng 9 cho lệnh `bne`. Để đọc mã, đổi từng chữ số hexa thành 4 bit rồi tách theo các trường 6, 5, 5 và 16 bit.

Bảng 9. Mã máy của lệnh bne trong Mã nguồn 3

Mục	Giá trị
Địa chỉ của lệnh `bne` (hexa)	0x00400010
Mã máy của `bne` (hexa)	0x16110002
`op`, `rs`, `rt` (thập phân)	op = 5, rs = 16, rt = 17
`immediate` (thập phân, có dấu)	2
`PC + 4 + immediate × 4` (hexa)	0x0040001c
Địa chỉ của nhãn `Else` trong Text Segment (hexa)	0x0040001c

Câu 2.3 (Phân tích) So sánh `PC + 4 + immediate × 4` với địa chỉ của nhãn `Else`. Vì sao `immediate` đếm theo số lệnh (từ 4 byte) mà không đếm theo byte?

Giá trị tính theo công thức PC + 4 + immediate * 4 hoàn toàn trùng khớp với địa chỉ của nhãn Else (0x0040001c).
Trường immediate đếm theo số lệnh thay vì byte vì trong MIPS, mọi lệnh đều có độ dài cố định 4 byte và nằm tại địa chỉ là bội số của. Việc quy ước immediate đếm theo đơn vị từ cho phép mở rộng phạm vi nhảy của 16-bit immediate lên gấp 4 lần.

Bước 2.4 – Phân bậc điện năng

Hoàn thành Mã nguồn 5: chương trình đọc điện năng (kWh) từ bàn phím, xác định bậc theo Bảng 6 và in `Bac: <bậc>`. Gợi ý: kiểm tra lần lượt từ bậc thấp đến bậc cao; "nhỏ hơn hoặc bằng 100" tương đương "nhỏ hơn 101" nên có thể dùng `slti $t0, $s0, 101` rồi `bne`. Mỗi bậc cần một nhãn riêng và một lệnh `j` để không chạy tiếp sang bậc khác. Chạy chương trình với từng giá trị ở Bảng 10 (điền cột DỰ ĐOÁN trước khi chạy).

Mã nguồn 5

```
# Ma nguon 5 – phan bac dien nang
.data
prompt: .asciiz "Nhap dien nang (kWh): "
msg:    .asciiz "Bac: "
nl:     .asciiz "\n"

.text
.globl main
main:
  li  $v0, 4
  la  $a0, prompt
  syscall
  li  $v0, 5            # doc kWh, ket qua o $v0
  syscall
  add $s0, $v0, $zero   # $s0 = dien nang (kWh)
  # >>> xac dinh bac (1..4) theo Bang 6, dat vao $s1
  # >>> in "Bac: ", roi $s1, roi xuong dong
  li  $v0, 10
  syscall
```

Bảng 10. Kiểm thử phân bậc

Điện năng nhập (kWh)	Bậc DỰ ĐOÁN	Bậc QUAN SÁT
50	1	1
100	1	1
101	2	2
150	2	2
151	3	3
200	3	3
201	4	4

Câu 2.4 (Đánh giá) Một bạn viết điều kiện của bậc 1 là "điện năng < 100" thay vì "điện năng ≤ 100". Giá trị nào trong Bảng 10 phát hiện ra lỗi này? Vì sao khi kiểm thử nên chọn giá trị ở biên của mỗi bậc?

Giá trị phát hiện ra lỗi sai này trong Bảng 10 là 100. Khi điện năng bằng 100, theo đề bài phải thuộc Bậc 1, nhưng nếu viết điều kiện là điện năng < 100 thì giá trị 100 sẽ bị sai điều kiện và bị đẩy nhầm sang Bậc 2.
Khi kiểm thử nên chọn các giá trị tại biên của mỗi bậc vì các lỗi logic trong lập trình thường xuyên xảy ra ở toán tử so sánh (dùng < thay vì <=, hoặc > thay vì >=). Giá trị ở giữa khoảng thường luôn đúng, chỉ có các giá trị tại biên mới bộc lộ sự sai lệch giữa hai nhánh điều kiện.

THÍ NGHIỆM 3. DUYỆT MẢNG BẰNG VÒNG LẶP (4 điểm)

Tình huống

Cuối năm, công tơ phải duyệt cả 13 chỉ số: tính điện năng từng tháng, tổng điện năng cả năm và tháng tiêu thụ nhiều nhất. Viết riêng từng tháng như ở Thí nghiệm 1 không còn khả thi khi mảng dài hơn, nên chương trình phải lặp lại cùng một đoạn lệnh với chỉ số thay đổi.

Kiến thức nền: vòng lặp và truy cập phần tử theo chỉ số

Vòng lặp được xây dựng từ lệnh rẽ nhánh và lệnh nhảy: khởi tạo chỉ số; nhãn `Loop:`; kiểm tra điều kiện thoát (`beq` hoặc `bne` đến nhãn thoát); thân vòng lặp; tăng chỉ số bằng `addi`; `j Loop`; nhãn thoát.

Khi chỉ số `i` nằm trong thanh ghi, độ dời không còn là hằng số mà phải tính ở mỗi vòng: `sll $t1, $s2, 2` (`$t1 = 4 × i`), `add $t1, $t1, $s0` (cộng địa chỉ nền), rồi `lw $t0, 0($t1)`. Lệnh `sll` dịch trái 2 bit, tương đương nhân với 4.

Bước 3.1 – Đọc một vòng lặp (DỰ ĐOÁN, QUAN SÁT, GIẢI THÍCH)

Gõ Mã nguồn 6; chương trình in 13 chỉ số, mỗi chỉ số một dòng. Trước khi chạy, điền cột DỰ ĐOÁN của Bảng 11. Sau đó đặt breakpoint tại lệnh `lw`, chạy bằng Run → Go rồi tiếp tục Go cho đến lần dừng thứ sáu (`i = 5`). Tại lần dừng này, nhấn Run → Step đúng một lần để thực hiện lệnh `lw`, rồi đọc các thanh ghi trong cửa sổ Registers; các dòng còn lại quan sát sau khi chạy hết chương trình.

Mã nguồn 6

```
# Ma nguon 6 – duyet va in mang reading
.data
nl:      .asciiz "\n"
reading: .word 1000, 1095, 1205, 1345, 1510, 1715, 1955
         .word 2210, 2440, 2625, 2775, 2895, 2995

.text
.globl main
main:
  la   $s0, reading     # $s0 = dia chi nen
  addi $s2, $zero, 0    # i = 0
  addi $s3, $zero, 13   # n = 13
Loop:
  beq  $s2, $s3, Done   # i == n thi thoat
  sll  $t1, $s2, 2      # $t1 = 4 * i
  add  $t1, $t1, $s0    # $t1 = dia chi reading[i]
  lw   $a0, 0($t1)      # $a0 = reading[i]
  li   $v0, 1
  syscall
  li   $v0, 4
  la   $a0, nl
  syscall
  addi $s2, $s2, 1      # i = i + 1
  j    Loop
Done:
  li   $v0, 10
  syscall
```

Bảng 11. Dự đoán và quan sát vòng lặp

Đại lượng	DỰ ĐOÁN	QUAN SÁT
Số dòng in ra khi chạy hết chương trình	13	13
Giá trị của `$s2` khi chương trình kết thúc	13	13
Khi `i = 5`, sau lệnh `lw`: `$t1 - $s0` (thập phân)	20	20
Khi `i = 5`, sau lệnh `lw`: giá trị của `$a0` (thập phân)	1715	1715

Câu 3.1 (Hiểu) Viết vòng lặp của Mã nguồn 6 dưới dạng một câu lệnh `for` của ngôn ngữ cấp cao, rồi chỉ ra các lệnh hợp ngữ thực hiện: khởi tạo, kiểm tra điều kiện và cập nhật chỉ số.

Vòng lặp tương đương trong ngôn ngữ cấp cao:
```
for (int i = 0; i < 13; i++) {
   printf("%d\n", reading[i]);
}
```
Các lệnh hợp ngữ thực hiện từng thành phần:
Khởi tạo: addi $s2, $zero, 0
Kiểm tra điều kiện: beq $s2, $s3, Done (kết hợp với addi $s3, $zero, 13)
Cập nhật chỉ số: addi $s2, $s2, 1

Bước 3.2 – Điện năng từng tháng

Tạo tệp mới, gõ Mã nguồn 7 (phần dữ liệu) rồi hoàn thành Mã nguồn 8 (phần lệnh). Với mỗi tháng `i` từ 0 đến 11, chương trình tính `usage[i] = reading[i+1] - reading[i]`, ghi vào mảng `usage` và in một dòng dạng `Thang <i+1>: <điện năng>`. Dùng `$s0`–`$s3` đúng như ghi chú trong mã khung và dùng `$t1`–`$t7` làm thanh ghi tạm.

Mã nguồn 7 (phần dữ liệu)

```
.data
m1:      .asciiz "Thang "
m2:      .asciiz ": "
mt:      .asciiz "Tong: "
mm:      .asciiz "Thang cao nhat: "
lp:      .asciiz " ("
rp:      .asciiz ")"
nl:      .asciiz "\n"
reading: .word 1000, 1095, 1205, 1345, 1510, 1715, 1955
         .word 2210, 2440, 2625, 2775, 2895, 2995
usage:   .space 48
```

Mã nguồn 8 (phần lệnh)

```
# Ma nguon 8 – dien nang tung thang
.text
.globl main
main:
  la   $s0, reading     # $s0 = dia chi nen reading
  la   $s1, usage       # $s1 = dia chi nen usage
  addi $s2, $zero, 0    # i = 0
  addi $s3, $zero, 12   # n = 12 thang
Loop:
  # >>> thoat neu i == n
  # >>> tinh dia chi reading[i]
  # >>> $t3 = reading[i], $t4 = reading[i+1]
  # >>> $t5 = $t4 - $t3 (dien nang thang i+1)
  # >>> tinh dia chi usage[i], luu $t5 vao usage[i]
  # >>> in "Thang ", i+1, ": ", $t5, xuong dong
  # >>> i = i + 1, quay lai Loop
Done:
  li   $v0, 10
  syscall
```

> **Kiểm tra kết quả**
> Với dữ liệu ở Mã nguồn 7, dòng đầu tiên phải là `Thang 1: 95` và dòng cuối cùng là `Thang 12: 100`.

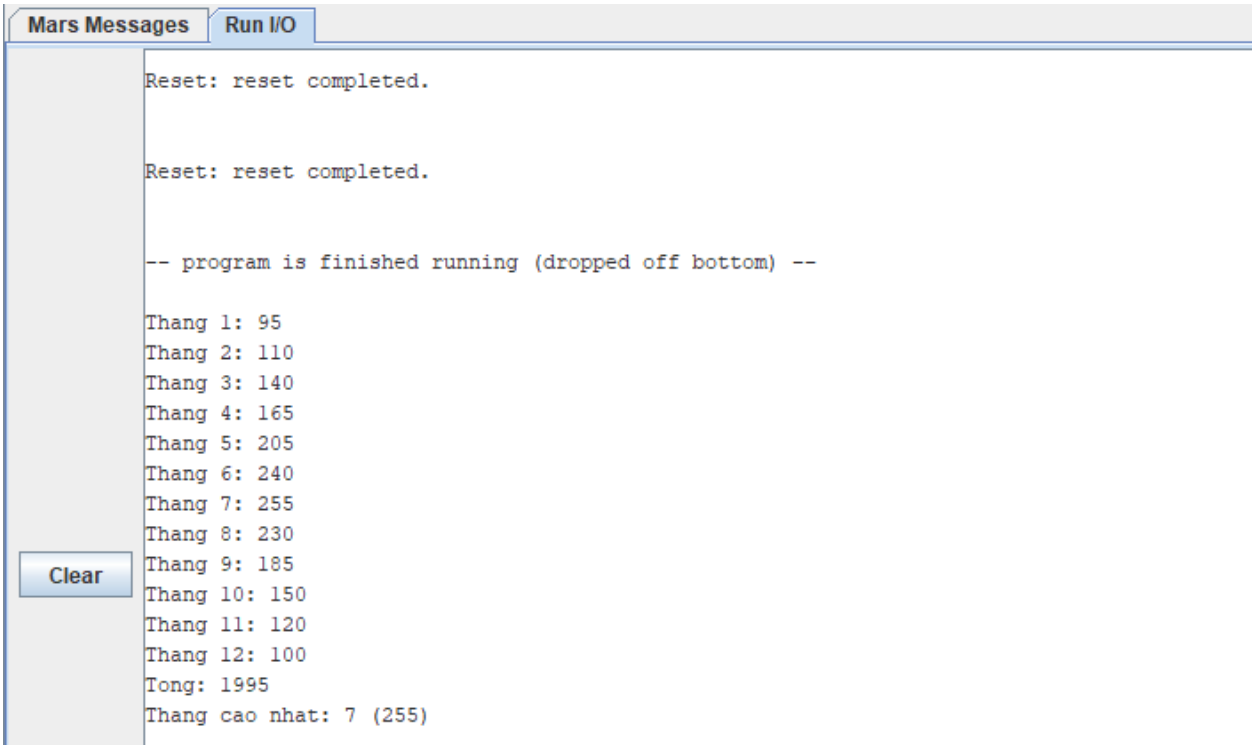
Bước 3.3 – Tổng điện năng và tháng cao nhất

Mở rộng chương trình: cộng dồn điện năng các tháng vào `$s4`; ghi nhớ điện năng lớn nhất vào `$s5` và tháng tương ứng vào `$s6`; sau vòng lặp, in thêm hai dòng `Tong: <tổng>` và `Thang cao nhat: <tháng> (<điện năng>)`. Gợi ý: so sánh điện năng của tháng hiện tại với `$s5` bằng `slt`, chỉ cập nhật `$s5` và `$s6` khi điện năng lớn hơn. Tổng điện năng cả năm cũng bằng `reading[12] − reading[0]`; hãy dùng đẳng thức này để tự kiểm tra.

> **Kiểm tra kết quả**
> Với dữ liệu ở Mã nguồn 7, hai dòng cuối phải là `Tong: 1995` và `Thang cao nhat: 7 (255)`.

Ảnh chụp khung Run I/O sau khi chạy chương trình hoàn chỉnh với dữ liệu ở Mã nguồn 7:

Dán ảnh chụp màn hình vào đây



Bước 3.4 – Dữ liệu riêng của sinh viên

Gọi D là chữ số cuối cùng của MSSV. Thay 13 số trong hai dòng `.word` của `reading` bằng hàng tương ứng ở Bảng 12. Điền cột DỰ ĐOÁN của Bảng 13 (điện năng tháng 1 và tổng cả năm, tính tay từ chỉ số), chạy chương trình rồi điền cột QUAN SÁT.

Bảng 12. Chỉ số riêng (D = chữ số cuối cùng của MSSV)

D	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]	[11]	[12]
0	1000	1095	1205	1345	1510	1715	1955	2210	2440	2625	2775	2895	2995
1	1020	1131	1272	1438	1644	1885	2141	2372	2558	2709	2830	2931	3027
2	1040	1182	1349	1556	1798	2055	2287	2474	2626	2748	2850	2947	3059
3	1060	1228	1436	1679	1937	2170	2358	2511	2634	2737	2835	2948	3091
4	1080	1289	1533	1792	2026	2215	2369	2493	2597	2696	2810	2954	3123
5	1100	1345	1605	1840	2030	2185	2310	2415	2515	2630	2775	2945	3155
6	1120	1381	1617	1808	1964	2090	2196	2297	2413	2559	2730	2941	3187
7	1140	1377	1569	1726	1853	1960	2062	2179	2326	2498	2710	2957	3219
8	1160	1353	1511	1639	1747	1850	1968	2116	2289	2502	2750	3013	3251
9	1180	1339	1468	1577	1681	1800	1949	2123	2337	2586	2850	3089	3283

Bảng 13. Kết quả với chỉ số riêng

D của tôi	Mục	DỰ ĐOÁN	QUAN SÁT
9	Điện năng tháng 1 (kWh)	159	159
9	Tổng điện năng cả năm (kWh)	2103	2103
9	Tháng cao nhất (tháng, kWh)	–	Tháng 10, 264 kWh

Bước 3.5 – Cố ý làm sai

Với mỗi thí nghiệm trong Bảng 14, dùng dữ liệu ở Mã nguồn 7. DỰ ĐOÁN dòng `Thang` cuối cùng được in và dòng `Tong`, thực hiện thay đổi, chạy chương trình và QUAN SÁT. Gợi ý cho (a): xác định địa chỉ của `reading[13]` và xem địa chỉ đó thuộc nhãn nào. Khôi phục chương trình sau mỗi thí nghiệm.

Bảng 14. Các thí nghiệm có kiểm soát

Thí nghiệm	DỰ ĐOÁN	QUAN SÁT
(a) Đổi số lần lặp từ 12 thành 13 (`addi $s3, $zero, 13`).	In thêm dòng Thang 13: -2900 (hoặc giá trị âm lớn do lấy usage[0] - reading[12] = 95 - 2995), dòng Tong: -905.	Dòng tháng cuối cùng in Thang 13: -2900, dòng tổng in Tong: -905.
(b) Đổi số lần lặp từ 12 thành 11 (`addi $s3, $zero, 11`).	Dòng tháng cuối cùng in ra là Thang 11: 120, dòng tổng in Tong: 1895 (thiếu tháng 12 với 100 kWh).	Dòng tháng cuối cùng in Thang 11: 120, dòng tổng in Tong: 1895.

Câu 3.2 (Phân tích) Giải thích kết quả (a): chương trình đã đọc giá trị nào làm `reading[13]`, vì sao MARS không báo lỗi, và điều này cho biết gì về việc phần cứng có kiểm tra giới hạn của mảng hay không?

Ở thí nghiệm (a), khi i = 12, chương trình truy xuất phần tử vượt biên mảng reading là reading[13]. Địa chỉ của reading[13] chính là 0x10010034, đây chính là địa chỉ bắt đầu của mảng usage (cụ thể là usage[0], lúc này đã được tính và có giá trị là 95). Do đó, phép trừ được thực hiện là: usage[0] - reading[12] = 95 - 2995 = -2900.
MARS không báo lỗi vì địa chỉ 0x10010034 hoàn toàn nằm trong vùng nhớ hợp lệ của Data Segment được cấp phát cho tiến trình và vẫn được căn lề biên từ 4 byte.
Điều này chứng minh rằng phần cứng CPU MIPS không hề có cơ chế tự động kiểm tra giới hạn mảng. Trách nhiệm kiểm soát chỉ số nằm ngoài biên mảng hoàn toàn thuộc về lập trình viên hoặc trình biên dịch cấp cao.

MỞ RỘNG. ĐẾM SỐ THÁNG THEO BẬC GIÁ (+1 điểm)

Thêm vào phần `.data` một mảng `cnt` gồm 4 từ khởi tạo bằng 0 (đặt sau `usage`). Duyệt lại mảng `usage`; với mỗi tháng, xác định bậc theo Bảng 6 (dùng lại cách ở Bước 2.4) rồi tăng phần tử tương ứng của `cnt` thêm 1 bằng ba lệnh `lw`, `addi`, `sw`. Cuối cùng in bốn dòng dạng `Bac <k>: <số tháng>`. Với dữ liệu ở Mã nguồn 7, số tháng của bốn bậc lần lượt là 2, 4, 2 và 4. Lưu chương trình vào tệp `IT012_Lab04_<MSSV>_MR.asm`.

Câu MR.1 (Đánh giá) Tiền điện của một tháng bằng điện năng nhân với đơn giá, nhưng bài này chưa dùng phép nhân. Đề xuất cách tính tích `a × b` chỉ bằng `add`, rẽ nhánh và vòng lặp, ước lượng số lần lặp khi điện năng là 255 kWh và nêu một nhược điểm của cách này.

Đề xuất cách tính tích a x b: Khởi tạo một thanh ghi chứa kết quả tích = 0. Dùng một vòng lặp chạy b lần, mỗi vòng lặp thực hiện cộng dồn a vào biến tích bằng lệnh add. Sau b lần lặp, biến tích sẽ chứa giá trị a x b.
Ước lượng số lần lặp: Nếu điện năng là 255 kWh và ta chọn cộng dồn đơn giá 255 lần thì vòng lặp cần thực hiện đúng 255 lần lặp.
Nhược điểm của cách này: Tốc độ thực thi rất chậm, độ phức tạp thuật toán là O(b), thời gian thực thi tỷ lệ tuyến tính với độ lớn của toán hạng. Khi nhân các số có giá trị lớn, CPU phải tốn rất nhiều chu kỳ thực hiện vòng lặp thay vì chỉ mất 1 chu kỳ như khi dùng bộ nhân phần cứng chuyên dụng.

PHẦN 4. TỔNG KẾT VÀ NỘP BÀI (1,5 điểm)

Câu 4.1 (Vận dụng) `$s0` chứa địa chỉ nền của mảng A các số nguyên (mỗi phần tử một từ). Viết các lệnh thực hiện `A[7] = A[2] + A[5]`.

```
lw  $t0, 8($s0)      # $t0 = A[2]  (offset = 2 * 4 = 8)
lw  $t1, 20($s0)     # $t1 = A[5]  (offset = 5 * 4 = 20)
add $t2, $t0, $t1    # $t2 = A[2] + A[5]
sw  $t2, 28($s0)     # A[7] = $t2  (offset = 7 * 4 = 28)
```

Câu 4.2 (Vận dụng) Dịch câu lệnh `if (x > y) z = 1; else z = 0;` sang MIPS, với x, y, z lần lượt nằm trong `$t0`, `$t1`, `$t2`.

```
slt  $t3, $t1, $t0   # neu y < x (tuc x > y) thi $t3 = 1, nguoc lai $t3 = 0
  beq  $t3, $zero, Else
  addi $t2, $zero, 1   # z = 1
  j    Exit
Else:
  addi $t2, $zero, 0   # z = 0
Exit:
```

Câu 4.3 (Đánh giá) Có hai cách duyệt mảng: (1) mỗi vòng tính địa chỉ phần tử bằng `sll` và `add` như ở Mã nguồn 6; (2) dùng một thanh ghi con trỏ và mỗi vòng cộng 4 vào con trỏ bằng `addi`. Mỗi cách cần bao nhiêu lệnh để xác định địa chỉ phần tử trong một vòng lặp? Với cùng CPI và chu kỳ xung nhịp, cách nào cho thời gian thực thi nhỏ hơn theo công thức thời gian CPU = số lệnh × CPI × chu kỳ xung nhịp? Vì sao?

Số lệnh xác định địa chỉ trong mỗi vòng lặp:
Cách 1 (dùng chỉ số): Cần 2 lệnh trong mỗi vòng lặp (sll $t1, $s2, 2 để tính độ dời và add $t1, $t1, $s0 để cộng vào địa chỉ nền).

Cách 2 (dùng con trỏ): Chỉ cần 1 lệnh (addi $ptr, $ptr, 4 ở cuối vòng lặp để chuyển con trỏ sang phần tử kế tiếp).
So sánh thời gian thực thi:
Cách 2 cho thời gian thực thi nhỏ hơn.

Giải thích: Theo công thức Thời gian CPU = Số lệnh x CPI x Chu kỳ xung nhịp. Do CPI và chu kỳ xung nhịp là như nhau, việc cách 2 tiết kiệm được 1 lệnh trong mỗi lần lặp làm tổng số lệnh thực thi giảm đáng kể (đặc biệt khi kích thước mảng lớn), từ đó giảm tổng thời gian CPU tiêu thụ.

Bảng 15. Danh sách kiểm tra khi nộp bài

☑	Nội dung
☑	Các tệp `IT012_Lab04_<MSSV>_TN1.asm`, `IT012_Lab04_<MSSV>_TN2.asm`, `IT012_Lab04_<MSSV>_TN3.asm` (và `IT012_Lab04_<MSSV>_MR.asm` nếu làm phần mở rộng) đã được lưu đúng tên.
☑	Mỗi tệp bắt đầu bằng khối chú thích (họ tên, MSSV, mục đích) và mọi chương trình đều assemble không lỗi.
☑	Mọi dấu `# >>>` đã được thay bằng mã của sinh viên và kế hoạch dùng thanh ghi được ghi trong chú thích.
☑	Tài liệu này đã hoàn chỉnh: mọi ô nền vàng đã được điền và mọi khung nét đứt đã có ảnh chụp màn hình.
☑	Các tệp `.asm` đã được đẩy lên kho mã GitHub của sinh viên, và liên kết đến kho mã đã được ghi vào bảng thông tin ở trang đầu.
☑	Tài liệu và liên kết kho mã được nộp theo hình thức giảng viên thông báo.

PHỤ LỤC. TỔNG HỢP CÁC LỆNH

Bảng 16. Các lệnh và chỉ thị dùng trong bài thực hành

Lệnh	Ví dụ	Ý nghĩa
`lw`	`lw $t0, 8($s0)`	`$t0 = Mem[$s0 + 8]`
`sw`	`sw $t0, 8($s0)`	`Mem[$s0 + 8] = $t0`
`sll`	`sll $t1, $s2, 2`	`$t1 = $s2` dịch trái 2 bit (nhân với 4)
`slt`	`slt $t0, $s0, $s1`	`$t0 = 1` nếu `$s0 < $s1`, ngược lại `$t0 = 0`
`slti`	`slti $t0, $s0, 101`	`$t0 = 1` nếu `$s0 < 101`, ngược lại `$t0 = 0`
`beq`	`beq $s0, $s1, L`	nhảy đến `L` nếu `$s0 == $s1`
`bne`	`bne $s0, $s1, L`	nhảy đến `L` nếu `$s0 != $s1`
`j`	`j L`	nhảy đến `L`
`blt`, `bge` (lệnh giả)	`blt $t0, $t1, L`	nhảy đến `L` nếu `$t0 < $t1` (hoặc >= đối với `bge`); assembler mở rộng thành lệnh thật
`.word`	`a: .word 3, 5, 7`	đặt các số 32 bit liên tiếp; nhãn `a` là địa chỉ nền
`.space`	`b: .space 48`	chừa 48 byte trống (12 từ)